



Chapter S4C: Web App CSS

CONTENT

No.	Topic	Syllabus Guide
1	Why Cascading Style Sheets	4.2
2	Anatomy of a CSS	
3	Your Own Style Sheet	
4	Types of Selectors	
	4.1 Element Selectors	
	4.2 id Selectors	
	4.3 class Selectors	
	4.4 Descendent Selectors	
5	Common Properties	4.2
6	Box Model Properties	
	6.1 Block and Inline Appearance	
	6.2 Margin, Border, Padding, Width and Height	
7	Typography Properties	



1. Why Cascading Style Sheets

Web pages created using HTML alone often look plain. Cascading Style Sheets (CSS) are used to improve the appearance of web pages without changing their structure.

This follows the principle of **separation of concerns**, where different aspects of a program are handled independently. In web design:

- HTML defines the structure (content and organisation)
- CSS controls the presentation (appearance and layout)

By keeping structure and presentation separate, we can modify one without affecting the other. For example, one person can manage the content while another designs the appearance. This also allows HTML and CSS to be specialised for their respective purposes.

A good demonstration is CSS Zen Garden (<https://csszengarden.com/>), where different CSS style sheets are applied to the same HTML file to produce completely different designs.

2. Anatomy of a CSS

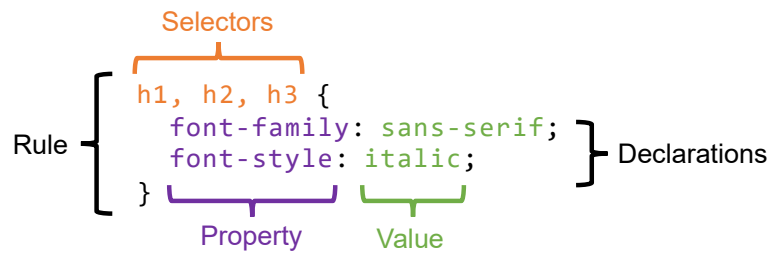
Open Google Chrome, visit www.example.com and view page source (Ctrl-U). Notice that the first part of the HTML document includes some CSS inside a `<style>` tag:

```
<style type="text/css">
body {
  background-color: #f0f0f2;
  margin: 0;
  padding: 0;
  font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;
}
div {
  width: 600px;
  margin: 5em auto;
  padding: 2em;
  background-color: #fdfdff;
  border-radius: 0.5em;
  box-shadow: 2px 3px 7px 2px rgba(0,0,0,0.02);
}
a:link, a:visited {
  color: #38488f;
  text-decoration: none;
}
@media (max-width: 700px) {
  div {
    margin: 0 auto;
    width: auto;
  }
}
</style>
```

For simplicity, this web page has chosen to include its style sheet inside the HTML file using a `<style>` tag. However, in general we write our style sheet in a separate `.css` file and link it to a HTML file using a `<link>` tag.



Focusing on the CSS between the `<style>` and `</style>` tags, we see that the style sheet is made of multiple **rules**. Each rule starts with one or more **selectors** separated by commas, followed by curly braces surrounding a number of **declarations**. These terms are illustrated for a single rule as follows:



Each declaration is made of two parts: a **property** name and one or more **values** separated by spaces. Multiple declarations in a rule are separated by semicolons.

<pre>body { margin: 0; padding: 0; } h1, h2, #danger { border: 1px solid red; }</pre>	<pre>body { background: white; color: black; }</pre>
--	--

3. Your Own Style Sheet

Like HTML documents, CSS style sheets are just text files with a `.css` extension. Open a text editor (such as Notepad++), create a new file and save it in your user directory as `example.css`. Then enter the following style sheet:

```
h1 { color: red; font-family: sans-serif; }  
p { color: blue; font-style: italic; }
```

To use a stylesheet, we need an accompanying document. Create a new file and save it as `example.html` in the same directory as `example.css`. Then enter the following HTML document:

```
<!doctype html>  
<html>  
  <head>  
    <title>CSS Example</title>  
    <link rel="stylesheet" href="example.css">  
  </head>  
  <body>  
    <h1>CSS Example</h1>  
    <p>This is an example of using CSS to style a web page.</p>  
  </body>  
</html>
```



Notice that a `<link>` tag has been added under the `head` element. The `rel` attribute (short for "relationship") has a value of "stylesheet", which indicates that the `href` attribute (short for "hyperlink") contains the URL of a CSS style sheet. This is how HTML documents are normally associated with style sheets. Now, double-click `example.html` in Windows Explorer to open it in a web browser.

You should see something similar to the following:

Congratulations! You have just used CSS to control the presentation of a web page.

CSS Example

This is an example of using CSS to style a web page.

Exercise 1

Create a HTML document and CSS style sheet using the following directory structure and contents. Open `index.html` and add a `<link>` tag so that the web page is styled using `colours.css`. Use relative paths so the style sheet is always linked correctly even if the `colourful` directory and its contents are moved to another location.



colours.css

```
body { background: cyan; }
h1 { color: blue; }
p {
    background: yellow;
    color: red;
}
```

index.html

```
<!doctype html>
<html>
  <head>
    <title>Colourful</title>
    <link .....>
  </head>
  <body>
    <h1>Welcome</h1>
    <p>This is a colourful document!</p>
  </body>
</html>
```



4. Types of Selectors

The elements which are affected by each CSS rule are determined by the selectors at the start of that rule. Each selector picks out (i.e., selects) a set of elements from the HTML document to be affected by the rule's declarations. Selectors are organised into four groups: *element selectors*, *id selectors*, *class selectors* and *descendent selectors*.

4.1 Element Selectors

An element selector picks out *all* elements of a particular type from the HTML document. To use an element selector, we simply enter the name of a tag or element type (e.g., `body`, `h1`, `table`, etc.). For instance, the second rule in `example.css` uses an element selector for `p` elements:

```
p { color: blue; font-style: italic; }
```

Based on this rule, all `p` elements on the web page will appear as blue italic text.

Sometimes, however, we do not want to select *all* the elements of a particular type. In such cases, we can fine-tune our selection by making use of two special attributes that are valid for all HTML tags: `id` and `class`.

4.2 id Selectors

An **id selector** targets a unique element using its `id` attribute. In CSS, it's written as a **hash symbol (#)** followed by the ID name. Because IDs must be unique within a page, this selector always picks a **single element**.

id-example.html

```
<!doctype html>
<html>
  <head>
    <title>ID Selectors Example</title>
    <link rel="stylesheet" href="id-example.css">
  </head>
  <body>
    <p>This is a normal paragraph.</p>
    <p id="special">This paragraph is special.</p>
    <p>This is a normal paragraph.</p>
  </body>
</html>
```

id-example.css

```
#special { color: red; }
```



If we open `id-example.html`, we see that only the second `p` element with an `id` of `special` is formatted as red.

This is a normal paragraph.

This paragraph is special.

This is a normal paragraph.

4.3 class Selectors

A **class selector** targets all elements with a specific class attribute. In CSS, use a **period** (`.`) followed by the class name. Unlike IDs, classes can be reused on multiple elements across a page.

class-example.html

```
<!doctype html>
<html>
  <head>
    <title>Class Selectors Example</title>
    <link rel="stylesheet" href="class-example.css">
  </head>
  <body>
    <p>This is the main paragraph.</p>
    <p class="info major">This is some major information.</p>
    <p class="info minor">This is some minor information.</p>
  </body>
</html>
```

class-example.css

```
.info { color: silver; }
```

If we open `class-example.html`, we see that only the second and third `p` elements with a class of `info` are formatted as silver. These `p` elements also have additional classes (i.e., `"major"` and `"minor"` respectively), but these have no effect as they do not match any of the selectors used in the style sheet.

This is the main paragraph.

This is some major information.

This is some minor information.

Choosing between `id` and `class` attributes:

- **id (#):** A **unique** identifier for a single element. No spaces allowed; cannot be reused on the same page.
- **class (.):** A **reusable** identifier for one or many elements. Can include a space-separated list of multiple classes.



4.4 Descendent Selectors

Sometimes, element selectors, id selectors and class selectors are not sufficient, and it is necessary to select an element only if the element has a parent element that matches another selector. This can be achieved using the descendent selector. To use a descendent selector, separate any two selectors using a space: the corresponding rule will only be applied for elements that match the selector on the right *and* have a parent element that matches the selector on the left. For instance, consider the following HTML and CSS files in the same folder:

Sometimes, basic selectors are not enough. You may want to style an element only when it is nested inside another specific element. This is done with a **descendant selector**, which is just a space between two names: Ancestor Descendant. It tells the browser: "Only apply this style to the **second** element if it is located anywhere inside the **first**"

descendent-example.html

```
<!doctype html>
<html>
  <head>
    <title>Descendent Selectors Example</title>
    <link rel="stylesheet" href="descendent-example.css">
  </head>
  <body>
    <h1>Heading with <i>Italics</i></h1>
    <p>This paragraph has <i>italics</i>.</p>
    <i>Bare italics</i>
  </body>
</html>
```

descendent-example.css

```
p i { color: red; }
```

Opening descendent-example.html shows that only `<i>` inside the `<p>` turns red. Other `<i>` elements remain unformatted because they are not nested within a paragraph as the selector requires.

Heading with *Italics*

This paragraph has *italics*.

Bare italics



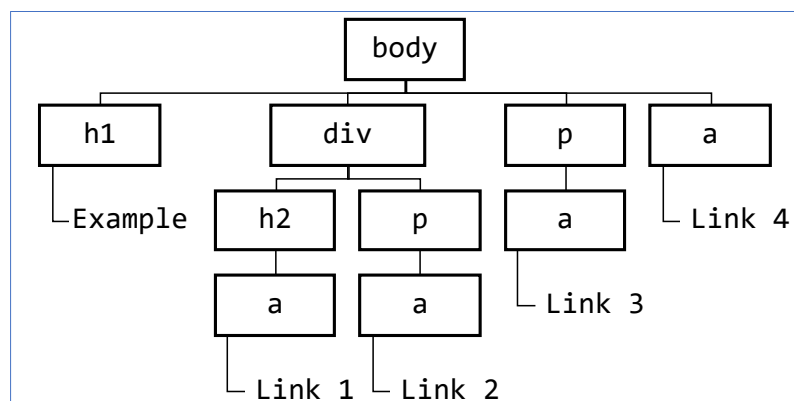
Examples of use of selectors

To further illustrate how the different selector types work, consider the following HTML document. In this example, we introduce a new `<div>` tag that is used as a generic element for grouping other elements.

multiple-example.html

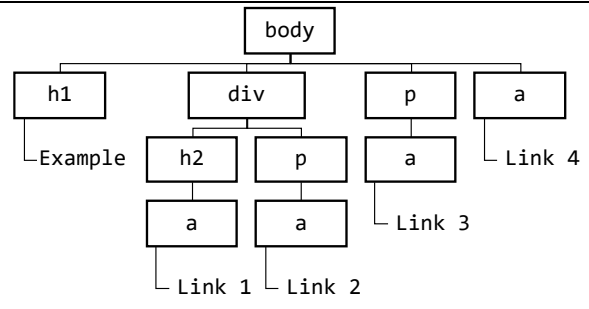
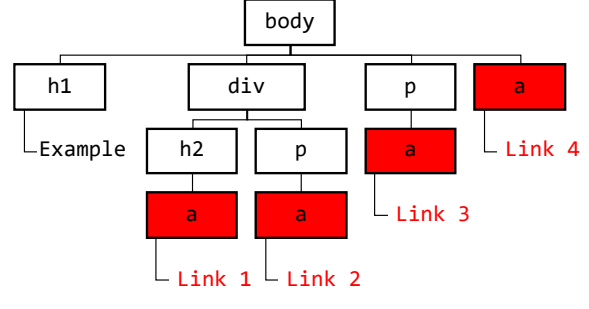
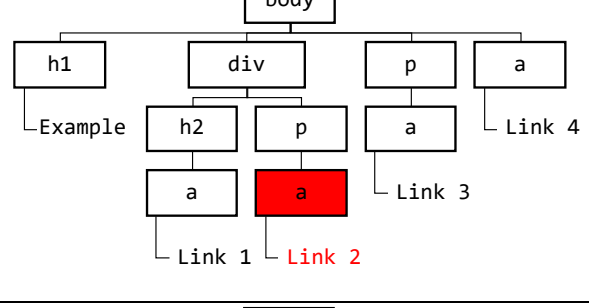
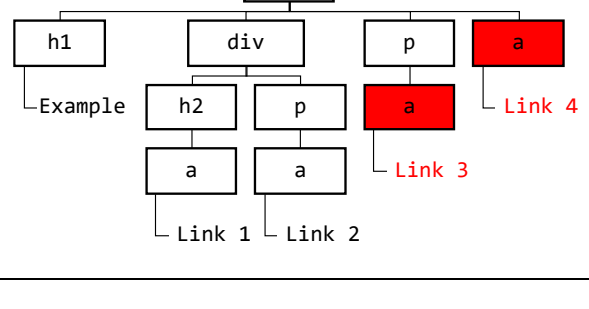
```
<!doctype html>
<html>
  <head><title>Example</title></head>
  <body>
    <h1>Example</h1>
    <div>
      <h2><a id="link1" href="link1.html">
        Link 1
      </a></h2>
      <p><a id="link2" href="link2.html">
        Link 2
      </a></p>
    </div>
    <p><a class="simple" href="link3.html">Link 3</a></p>
    <a class="simple" href="link4.html">Link 4</a>
  </body>
</html>
```

This HTML describes the following tree structure of elements:





Based on this HTML document, the following table shows which elements from the tree are matched for different CSS selectors:

CSS	Matched Elements	Result
<i>No style sheet</i>		Example Link 1 Link 2 Link 3 Link 4
<code>a { background: red; }</code>		Example Link 1 Link 2 Link 3 Link 4
<code>#link2 { background: red; }</code>		Example Link 1 Link 2 Link 3 Link 4
<code>.simple { background: red; }</code>		Example Link 1 Link 2 Link 3 Link 4



<pre>div a { background: red; }</pre>		Example Link 1 Link 2 Link 3 Link 4
---	--	--

If multiple rules conflict, the most **specific** selector wins. CSS uses a built-in formula to calculate this, but you can avoid confusion by writing clear, non-conflicting rules.

With selectors covered, let's explore the **three property groups**: common, box model, and typography.

Exercise 2

Use a text editor to create the following HTML document. Without changing the HTML, add a CSS file such that only the text "RED" is coloured in red:

exercise2.html

```
<!doctype html>  
<html>  
  <head>  
    <title>Selectors Quiz</title>  
    <link rel="stylesheet" href="selectors.css">  
  </head>  
  <body>  
    BLACK  
    <h1>RED</h1>  
    <p>BLACK <b>BLACK</b> <i>BLACK</i></p>  
    <p id="test1">BLACK <b>RED</b></p>  
    <p class="test2">BLACK <i>RED</i></p>  
  </body>  
</html>
```



5. Common Properties

Many properties, like background and color, work with *any element*. These allow you to set the background and text colours using specific colour names.

Here are some of the colour names that can be used with the background and color properties:

red	orange	yellow	green	blue	purple
black	gray	silver	white	transparent (no color)	

If a colour isn't named, use **RGB values** from 0 to 255. Written as `rgb(R, G, B)`, these represent the red, green, and blue components. For example, the following CSS sets the page background to a shade of pale yellow:

```
body { background: rgb(255, 255, 128); }
```

Colors can also be written as **hexadecimal codes** (`#RRGGBB`). Each two-digit pair represents the red, green, and blue components in hex (00 to FF). For example, the shade of pale yellow can also be written as:

```
body { background: #ffff80; }
```

Notice that `ff` is the integer 255 in hexadecimal, while `80` is the integer 128 in hexadecimal. Also note that hexadecimal digits are not case sensitive.

Besides colors, we can also use a repeating image as the background by using `url(PATH)`, where `PATH` is the path and filename of the background image. For instance, the following CSS uses `pattern.png` (located in the same folder with the style sheet) as a repeating background image for the page.

```
body { background: url(pattern.png); }
```

The following HTML and `pattern.png` image demonstrates this:

HTML	pattern.png	Result
<pre><body> <p>Hello, World</p> </body></pre>		



6. Box Model Properties

6.1 Block and Inline Appearance

HTML tags like `<h1>` and `<p>` are **block** elements, meaning they always start on a new line. Tags like `<b>` and `<i>` are **inline** elements, so they stay on the same line as the surrounding text.

HTML	Result
<code><h1>h1</h1></code> <code><p>p</p></code> <code>b</code> <code><i>i</i></code>	h1 p b <i>i</i>

display property

However, we can use the `display` property to override these defaults. For instance, the following CSS changes all the elements to use a block appearance instead:

CSS	Result
<code>h1, p, b, i { display: block; }</code>	h1 p b <i>i</i>

Similarly, the following CSS changes all the elements to use an inline appearance:

CSS	Result
<code>h1, p, b, i { display: inline; }</code>	h1 p b <i>i</i>

Setting `display: none;` hides the element entirely, removing it from the page layout as if it isn't there.

<div> and Tags

Sometimes, we need generic elements to group content for styling.

- `<div>` is a **block element** (starts on a new line)
- `<span>` is an **inline element** (stays within a line)

They are often used with **class** or **id** to apply CSS styles.



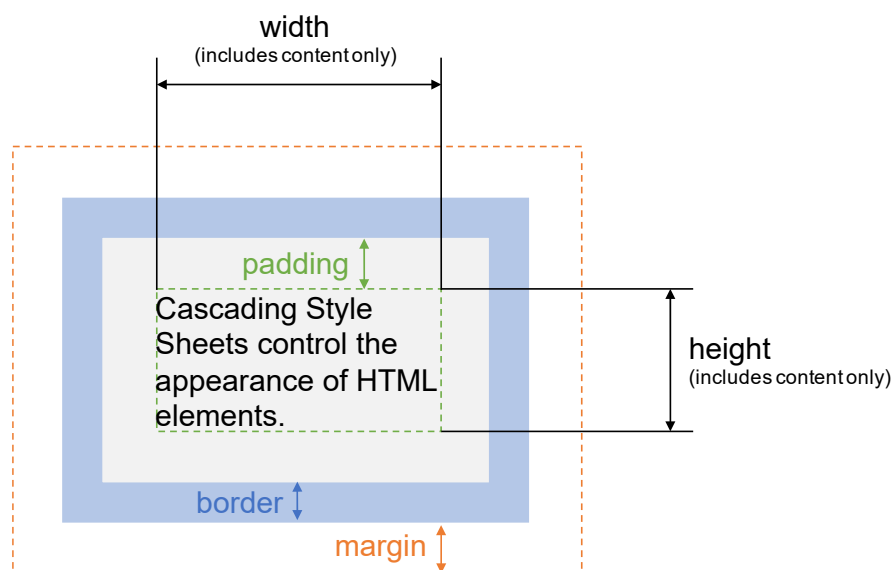
For instance, the following HTML document on Python syntax uses `<div>` and `<span>` tags so that Python's rules and keywords are marked out and styled:

HTML	CSS	Result
<pre><h1>Functions</h1> <div class="rule"> Python functions are defined using the def keyword. </div></pre>	<pre>.rule { background: silver; } .keyword { color: red; }</pre>	Functions Python functions are defined using the def keyword.

6.2 Margin, Border, Padding, Width and Height

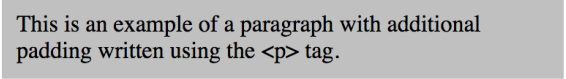
Block elements are displayed as boxes. The **box model** describes how spacing and size are controlled.

Property Name	Description
border	The edge surrounding the element
margin	Space outside the border
padding	Space between content and border
width	Width of the content area
height	Height of the content area

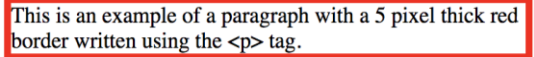




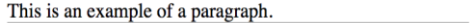
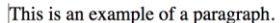
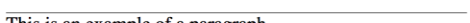
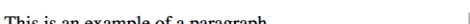
We usually specify sizes in pixels (px).

CSS	Result
<pre>p { background: silver; padding: 10px; }</pre>	 This is an example of a paragraph with additional padding written using the <p> tag.

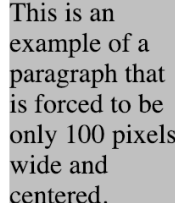
To draw a border, specify the thickness, style (e.g. solid), and colour. For example, the following CSS gives p elements a 5px red border:

CSS	Result
<pre>p { border: 5px solid red; }</pre>	 This is an example of a paragraph with a 5 pixel thick red border written using the <p> tag.

By default, margin, border, and padding affect all four sides. To target a specific side, append **-top**, **-bottom**, **-left**, or **-right** to the property (e.g., border-bottom).

CSS	Result
<pre>p { border-bottom: 1px solid gray; }</pre>	 This is an example of a paragraph.
<pre>p { border-left: 1px solid gray; }</pre>	 This is an example of a paragraph.
<pre>p { border-top: 1px solid gray; }</pre>	 This is an example of a paragraph.
<pre>p { border-right: 1px solid gray; }</pre>	 This is an example of a paragraph.

A block element can be centred horizontally by setting its left and right margins to **auto** and giving it a fixed width.

CSS	Result
<pre>p { background: silver; margin-left: auto; margin-right: auto; width: 100px; }</pre>	 This is an example of a paragraph that is forced to be only 100 pixels wide and centered.



Exercise 3

For each HTML document below, create a CSS file so that the completed web page appears as closely to the required appearance as possible.

HTML	Required Appearance
<pre><h1>Introduction</h1> <p>This is an example of using borders to make headings prettier.</p> <h1>Explanation</h1> <p>Each heading has a line below it that extends across the entire screen.</p></pre>	Introduction This is an example of using borders to make headings prettier. Explanation Each heading has a line below it that extends across the entire screen.
<pre><p>Printing documents can be a very tricky business. There are many things to look out for.</p> <p class="tip">Tip: Make sure the printer has paper before printing!</p> <p>However, if you get the process right, you will be rewarded with a marvelous- looking printout.</p></pre>	Printing documents can be a very tricky business. There are many things to look out for. Tip: Make sure the printer has paper before printing! However, if you get the process right, you will be rewarded with a marvelous-looking printout.
<pre><h1>Heading</h1> <p>This is an example of a paragraph that has both margin and padding set.</p></pre>	Heading This is an example of a paragraph that has both margin and padding set.



7. Typography Properties

The **font-family** property sets the text's typeface using a comma-separated list. If a font name contains spaces, enclose it in quotes (e.g., "Times New Roman"). The browser tries each font in order, falling back to a generic **serif** or **sans-serif** if none are installed.

The following examples illustrate how to set a serif or sans-serif font using CSS:

CSS	Result
<code>p { font-family: serif; }</code>	This is an example of a paragraph.
<code>p { font-family: sans-serif; }</code>	This is an example of a paragraph.

The **font-size** property can be used to specify text size in pixels.

CSS	Result
<code>p { font-size: 24px; }</code>	This is an example of a paragraph.

The **font-style** property toggles between normal and italic. Similarly, the **font-weight** property toggles between normal and bold.

CSS	Result
<code>p { font-style: italic; }</code>	<i>This is an example of a paragraph.</i>
<code>p { font-weight: bold; }</code>	This is an example of a paragraph.
<code>p { font-style: italic; font-weight: bold; }</code>	<i>This is an example of a paragraph.</i>

The **text-align** property specifies how the text is aligned within its container. The most common values for this property are left, center, right and justify:

CSS	Result
<code>p { text-align: left; }</code>	This is an example of a paragraph with enough content to see how text-align works.
<code>p { text-align: center; }</code>	This is an example of a paragraph with enough content to see how text-align works.
<code>p { text-align: right; }</code>	This is an example of a paragraph with enough content to see how text-align works.
<code>p { text-align: justify; }</code>	This is an example of a paragraph with enough content to see how text-align works.



The text-decoration property adds visual effects like underline, line-through (strikethrough), or none (commonly used to un-style links).

CSS	Result
<pre>p { text-decoration: underline; }</pre>	<u>This is an example of a paragraph.</u>
<pre>p { text-decoration: line-through; }</pre>	This is an example of a paragraph.

Exercise 4

For each HTML document below, create a CSS file so that the completed web page appears as closely to the required appearance as possible. However, there is no need to match the required appearance exactly, so if a font size is needed, use an approximate value.

HTML	Required Appearance
<pre><h1>Reminder</h1> <p>This is an important reminder to try new & different ideas.</p></pre>	Reminder This is an <u>important</u> reminder to try new & different ideas.
<pre><h1>Heading 1</h1> <h2>Heading 2</h2> <p>Paragraph</p></pre>	Heading 1 <i>Heading 2</i> Paragraph
<pre><p>This is a link to the root of your web site.</p></pre>	This is a <u>link</u> to the root of your web site.